Preparcial: central de envíos

Una mensajería recibe un CSV de paquetes preparados. El programa carga envíos válidos y calcula el costo de transportarlos por zona. Este proyecto se entrega para estudiar y ejecutar; no hay tareas de implementación para esta instancia.

Ejecución

Requiere JDK 21 o 25 y Maven 3.9.x. Desde esta carpeta:

```
mvn test
java -cp target/classes ar.edu.backend.envios.Main
java -cp target/classes ar.edu.backend.envios.Main datos/datos.csv
```

La primera ejecución de Maven puede requerir Internet. Una vez resueltas las dependencias, probar `mvn -o test` en la misma computadora. El programa no requiere dependencias externas en ejecución.

Contrato del archivo

UTF-8, encabezado exacto `id,zona,brutoGramos,embalajeGramos,estado`. Cada registro ocupa una línea con cinco campos separados por coma. Los campos no admiten comas, comillas ni saltos de línea internos; no hay campos entrecomillados. Se quitan espacios externos de cada campo. Es un formato de intercambio controlado, no un lector de todo CSV posible. El uso de `split(",", -1)` conserva los campos finales vacíos y es válido bajo este contrato.

Cada fila tiene identificador y zona no vacíos, peso bruto y peso del embalaje enteros en gramos, y estado. No se exige unicidad de identificadores; cada fila válida cuenta como un envío independiente. Las zonas son textos sensibles a mayúsculas.

El peso bruto debe estar entre 1 y 50000 g; el embalaje debe ser no negativo y menor al bruto. El peso neto es bruto menos embalaje y debe estar entre 1 y 30000 g. Estos cálculos y validaciones quedan cerca del objeto, en `Envio.desdeCampos` y su constructor. El constructor recibe peso **neto**.

El costo en pesos enteros es `500 + 200 × kilos facturables`. Cada fracción de kilo cuenta como un kilo completo: 1000 g cuestan 700 y 1001 g cuestan 900. No se usan números decimales ni redondeos monetarios.

Flujo y errores

El parser verifica primero la cantidad de columnas. Con ancho correcto, `CANCELADO` se descarta sin validar los restantes datos. `LISTO` se intenta convertir; cualquier otro estado es inválido. Una línea vacía es inválida. No se interrumpe la carga por errores de una fila: se registra el número físico de línea y el motivo. Un encabezado incorrecto aborta con `IllegalArgumentException`; un fallo de lectura propaga `IOException`. El encabezado no cuenta como fila leída.

`procesadas` significa filas aceptadas, no intentos de conversión. Se cumple `leídas = procesadas + descartadas + inválidas` y `objetos = procesadas`. Se conserva el orden de los objetos aceptados y de los diagnósticos.

Recorrido de lectura sugerido

1. Ejecutar `Main` y los tres grupos de tests.
2. Leer `Envio`: invariantes, conversión textual y costo derivado.
3. Seguir `ParserEnvios` y `ResultadoParseo`: clasificación y manejo de excepciones.
4. Leer `CentralEnvios`: copia defensiva, `Predicate<Envio>`, filtros, suma y agrupamiento.
5. Localizar casos testigo en el archivo y explicar su clasificación y costo; usar el programa para verificar los agregados del conjunto.

El archivo contiene 60 filas de datos más encabezado (61 líneas). Panorama esperado: 60 leídas, 39 procesadas, 6 descartadas y 15 inválidas; total 86100; zonas Centro 30700, Norte 28500, Sur 26900; ocho envíos de hasta 1000 g. Casos testigo: E01 (1000 g netos), E06 (30000 g), E07 (cancelado con peso cero), E09 (neto cero inválido), I013 (1 g) e I049 (30001 g, inválido).

Los getters se escriben explícitamente: no se agrega Lombok para minimizar dependencias y configuración del entorno de examen. ResultadoParseo también usa getters convencionales y conserva copias defensivas de sus listas. JUnit 5 es la única dependencia y se usa solamente en tests.